Amazon-style Proxy Bidding with No Reserve Price

EDSL shared-state experiment setup

October 10, 2026

Abstract

This standalone document specifies the shared-state implementation and planned execution for this archived llm-auction condition. It is a setup paper: no new inference has been run for this condition, so it reports no model outcomes.

1 Condition and protocol

Amazon-style Proxy Bidding with No Reserve Price. This condition uses 3 bidders, private values, and 100 configured auction repetitions with model gpt-4o at temperature 0.5. Bidders submit sealed bids concurrently against a shared snapshot. The state machine withholds bid records until all required bids arrive, then settles the auction in one close transition.

Item	Setup
Condition	amazon_reserve_0
Bidders	3
Value model	private
Configured repetitions	100
Model	gpt-4o
Temperature	0.5
Inference status	Not run; this file is a setup paper
Machine	Defined; diagram generated from EDSL Machine.show_diagram()
bottomrule	

Table 1: Experiment setup; no inference results are included.

2 Shared-state machine

Each repetition has its own scope in the shared-state map. The machine definition is $M = (C, X, A, V, \delta, \kappa)$: constants C, internal state X, action interface A, public view V, action transition δ, and close transition κ. Separate scopes ensure that a transition in auction r cannot alter auction r'. The state stores bid records $B = \{(i, s_i, v_i, c_i, b_i)\}$. At close they are sorted by $(-b_i, s_i)$; the configured order selects the settlement bid. In first-, second-, or third-price auctions only the winner pays that bid; in all-pay every bidder pays their own bid. Utility is value less payment.

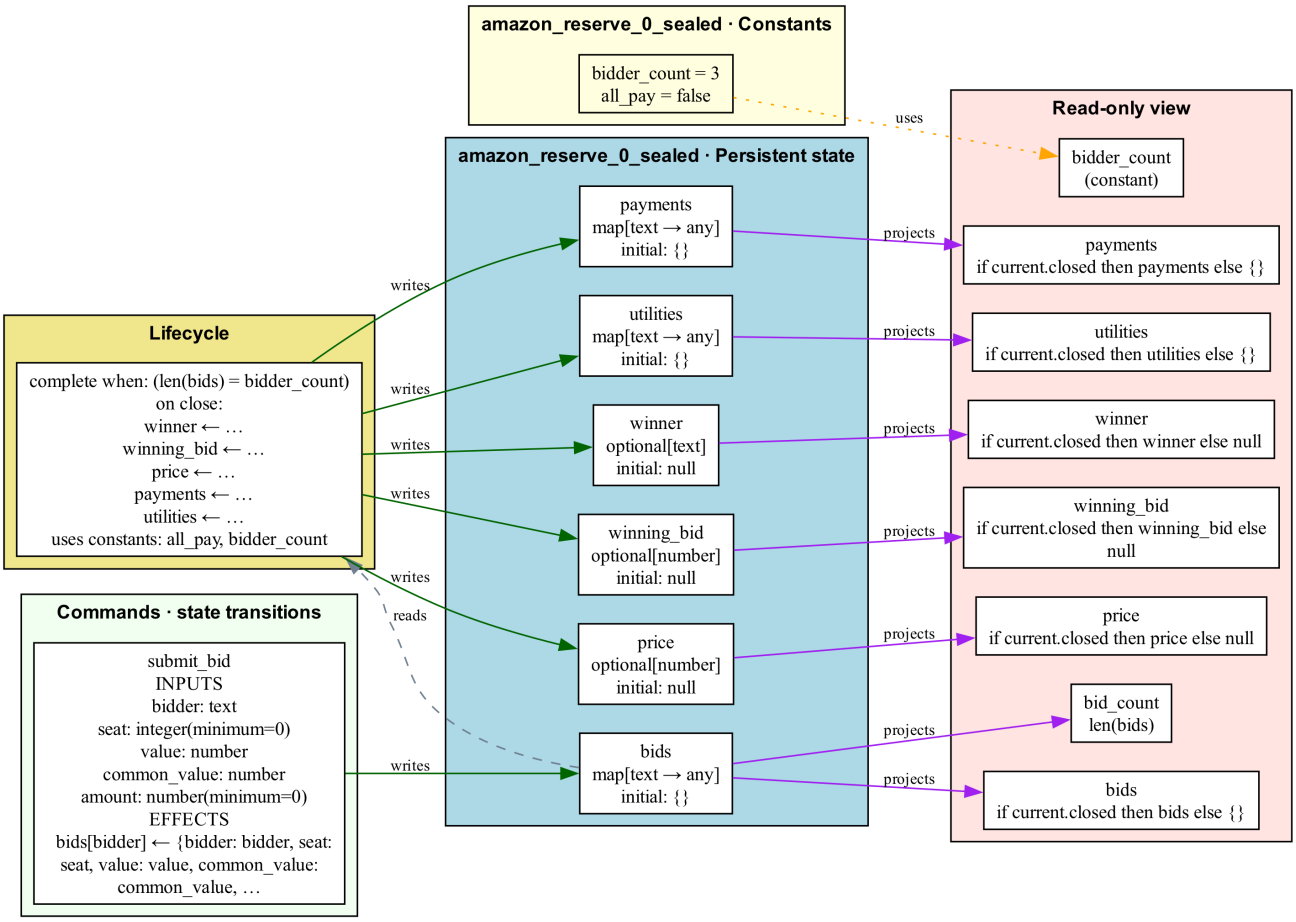


Figure 1: Diagram generated by EDSL's `Machine.show_diagram()`.

2.1 Python machine factory

The following is the implementation used to build this condition's machine.

```
def build_sealed_machine(config: dict[str, Any]) -> Machine:
    """Build a sealed-bid mechanism with config-specific constants and settlement."""
    auction, rule = config["auction"], config["rule"]
    count = auction["number_agents"]
    order = rule["price_order"]
    rank_index = {"first": 0, "second": 1, "third": 2, "allpay": 0}[order]
    ranked = reduce(
        "sort_records", field("bids").values(), fields=("amount", "seat"),
        descending=(True, False),
    )
    winner = ranked.first()
    winner_id = winner.get("bidder")
    settlement_bid = choose(
        ranked.length() > rank_index, ranked.at(rank_index).get("amount"), 0
    )
    value_field = "common_value" if rule["private_value"] == "common" else "value"
    payments = map_items(
        field("bids"), key="bidder", value="bid", key_expr=local("bidder"),
        value_expr=choose(
```

```
            constant("all_pay"), local("bid").get("amount"),
            choose(local("bidder") == winner_id, settlement_bid, 0),
        ),
    )
    utilities = map_items(
        field("bids"), key="bidder", value="bid", key_expr=local("bidder"),
        value_expr=local("bid").get(value_field) - payments.get(local("bidder")),
    )
    machine = Machine(
        name=f"{config['experiment']['name']}_sealed",
        constants={"bidder_count": count, "rank_index": rank_index,
                   "all_pay": order == "allpay", "price_order": order,
                   "value_type": rule["private_value"]},
        fields={
            "bids": state_field(StateType.map(), {}),
            "winner": state_field(StateType.optional(StateType.text()), None),
            "winning_bid": state_field(StateType.optional(StateType.number()), None),
            "price": state_field(StateType.optional(StateType.number()), None),
            "payments": state_field(StateType.map(), {}),
            "utilities": state_field(StateType.map(), {}),
        },
        commands={"submit_bid": Command(
            inputs={"bidder": StateType.text(), "seat": StateType.integer(minimum=0),
                    "value": StateType.number(), "common_value": StateType.number(),
                    "amount": StateType.number(minimum=0)},
            effects=(put("bids", arg("bidder"), record(
                bidder=arg("bidder"), seat=arg("seat"), value=arg("value"),
                common_value=arg("common_value"), amount=arg("amount"),
            )),),
        )},
        view={
            "bidder_count": constant("bidder_count"),
            "bid_count": field("bids").length(),
            "bids": choose(current("closed"), field("bids"), {}),
            "winner": choose(current("closed"), field("winner"), None),
            "winning_bid": choose(current("closed"), field("winning_bid"), None),
            "price": choose(current("closed"), field("price"), None),
            "payments": choose(current("closed"), field("payments"), {}),
            "utilities": choose(current("closed"), field("utilities"), {}),
        },
        complete_when=field("bids").length() == constant("bidder_count"),
        close_effects=(assign("winner", winner_id),
                       assign("winning_bid", winner.get("amount")),
                       assign("price", settlement_bid),
                       assign("payments", payments), assign("utilities", utilities)),
    )
    return machine
```

3 Launch command

The EDSL runner executes locally and uses the model service and key configured for this experiment. This setup step did not launch interviews. To run the configured number of repetitions:

```
python -m edsl_replication.shared_state_auctions \\
  configs/experiments/02\_05\_amazon\_reserve\_0.yaml \\
```

```
  --output edsl_replication/studies/amazon\_reserve\_0/raw_results.json
```

4 Results

Results are pending. After inference, the condition report will be rebuilt from the EDSL result object, interview answers, machine event history, and exit snapshots. No outcome estimates or charts are included in this setup paper.